Large Language Models for Social Science Research

Session 1. Setting Up an Agentic Research Project

Hugo Subtil

October 14, 2026

Before We Start

9:00 – 9:15

The idea of this session, and why it is debatable

An Economics Paper, Produced Autonomously

- An AI system writes economics papers on its own: question, data, code, results, text.
- Autonomous Policy Evaluation (APE), Social Catalyst Lab, University of Zurich.
- 900+ papers by spring 2026, ranked by LLM referees against published papers.

▶ Video, 2 minutes

Posted on LinkedIn by the project's author, February 2026

The Author's Own Reading: Two Worlds

World 1. This is real research. But it has nothing to do with my expertise. I cannot evaluate it. Some other human expert has to.

World 2. This is not real research. I just produced a negative externality by polluting the information environment.

His conclusion. In a world of machine-speed papers, we have a problem either way. Generation speed $\gg$ verification speed. This was never the case historically. Anybody, with no expertise, can now produce a paper that looks legitimate at first glance.

Open questions he lists: matching in the academic labour market, credit attribution. "I don't have the answers."

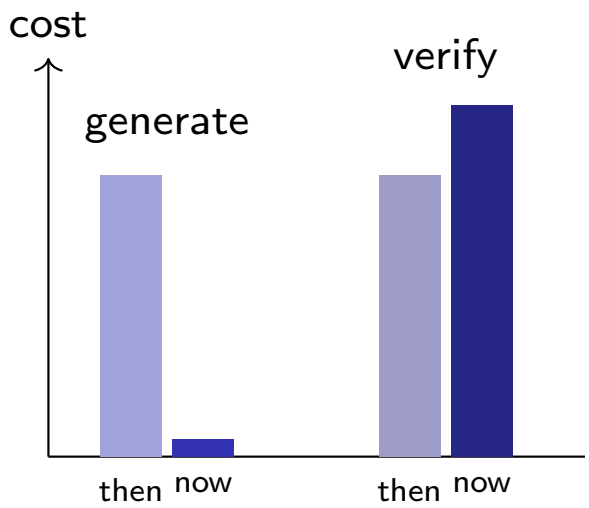
Neither do I. But we can push this a little further,
and it changes what we do in this room.

Pushing Further (1): The Bottleneck Moved

His two worlds have one consequence in common. Either way, the scarce resource is no longer production. It is qualified verification. The bottleneck moved from *doing* to *judging*.

Why did this never happen before?

- Verification was always slow. But generation was slow too.
- The cost of writing a paper was a filter: you did not write one you did not believe in.
- It was also a signal of competence (Spence 1973): costly, hence informative.
- Peer review was calibrated for a world where submitting cost a year.



Generating gets cheaper. Verifying gets *dearer*: you now check the pipeline, not only the text.

Pushing Further (2): Verification at Machine Speed

This is not hypothetical. People have already looked at the APE papers from the outside.

- **Cunningham (2026)** went through 651 APE papers: systematic p-hacking patterns.
- **Ning (2026)** compared 912 APE papers with 41 *AER* and *AEJ* papers: the human papers win 83% of head-to-head comparisons, and most of the gap is on the *idea* (originality, relevance, choice of design), not on execution.

The detail that matters for us. To verify 651 papers, Cunningham used a large language model. The APE tournament itself is judged by LLM referees.

Who verifies the verifier?

Verification at machine speed is possible. It is also the same machine. The chain has to end with a human who understands what was done.

Pushing Further (3): The Dependence Trap

The capacity to verify is produced by doing. If nobody does the thing any more, then within one cohort nobody can check the thing. And when nobody can check, everybody depends on whoever still can.

The Covid version. Countries that had stopped making basic goods (masks, paracetamol) found they could not make them when it mattered. Not because it is hard. Because they had handed the know-how to someone else.

The academic version. The senior PI who delegates everything to research assistants and stops checking. When it breaks, it is *their* name on the paper. Responsibility does not delegate.

An agent is a research assistant who never gets tired, never says "I am not sure" unless you ask, and bears no responsibility at all.

The Trade-off, and Why It Is Yours to Make

Short term, individual. Delegating everything is rational. You will produce more, faster. The productivity imperative is real, and your career is short.

Long term, collective. Expertise is a common-pool resource. Every person who stops understanding depletes it a little. Nobody feels it until it is gone.

And the feeling of productivity is not productivity. In a randomised trial (Becker et al. 2025), experienced open-source developers were 19% slower with AI tools on their own code, while believing they had been about 20% faster. A narrow setting, but keep the gap between the feeling and the measurement.

I am not going to pretend this trade-off is balanced.
It is your choice. Here is mine.

My Position, and Three Rules for This Workshop

Where I stand

I am not here to teach you the technique. I am here to show you how to delegate it. But I think we have a moral obligation to understand what we delegate. Otherwise we will not know how to do it, and when nobody knows, we all depend on someone else.

1. **You delegate execution, never understanding.** You must be able to explain every file in your project.
2. **Read before you accept.** Every output is a draft. Every session has a mandatory manual check.
3. **Stop when you no longer know what it is doing.** Supervising agents is work. Fatigue is the signal to stop, not to accept faster.

This is exactly why we do not start with a chat.
We start with a folder.

Today's Session Overview (9:00–12:00)

9:00–9:15 Before We Start (15 min)

9:15–9:30 An Agentic Research Project (15 min)

- Chat vs agent, why a folder, the template, guardrails

9:30–9:50 Installation Clinic (20 min)

- Worksheet, steps 0 and 1; problems go in the shared document

9:50–10:25 Live Demo (35 min), then a 15-min break

- The example project, from an empty folder to a first commit

10:40–11:40 Hands-on (60 min)

- The same thing, on your project: worksheet, steps 1 to 8

11:40–12:00 Debrief and Next Session (20 min)

An Agentic Research Project

9:15 – 9:30

From a chat that forgets to a folder that remembers

Course Repository



What is in it

- `template/`: the project folder you copy today
- `sessions/01/`: these slides, the worksheet
- `setup/`: installation guide and test
- `example-project/`: my project, session after session

To get it
Green "Code" button, then "Download ZIP".
Or: `git clone` the address above.

Shared document for errors and questions: link on the board.

`github.com/subtilhugo/uzh-llm-socsci-2026`

Chat vs Agent

A chat

- You paste; it answers; you copy. It sees what you paste, nothing else.
- It forgets everything when you close the tab.
- It cannot open your PDF, run your script, or look at your data.

An agent

- It works inside a folder on your machine.
- The loop: read files $\rightarrow$ run commands $\rightarrow$ write files $\rightarrow$ report.
- It asks permission before each action that changes something.
- Claude Code, Gemini CLI, Codex: same idea, different vendors.

The permission prompt is the only thing that separates an assistant from an unsupervised intern. Keep it on. Read it.

Why a Folder, Not a Chat

1. **Context.** The agent reads `CLAUDE.md` at the start of every session. You explain your project once, in a file, not in every conversation.

2. **Memory.** `PROJECT_HISTORY.md` says where the project stands. A fresh session can answer "where are we?" from the file alone. Nothing lives in the chat.

3. **Reproducibility.** Git, versioned prompts, scripts that regenerate outputs from raw data. In a year you will know what was done, from what, and by whom.

And one more: the folder is yours. Change tool tomorrow, the folder stays. The same `CLAUDE.md` works in the terminal, in Cursor or VS Code, in the desktop app. A chat history belongs to a vendor; a folder belongs to you.

Anatomy of the Project Template

`my-project/`	
`README.md`	what the project is
`CLAUDE.md`	how the agent behaves
`PROJECT_HISTORY.md`	where we stand
`literature/`	PDFs (read-only)
`notes/literature/`	one note per paper
`notes/`	memos, critiques
`data/raw/`	never modified
`data/processed/`	rebuilt by scripts
`code/`	numbered scripts
`outputs/`	figures, tables
`prompts/`	the prompts, versioned
`paper/`	drafts
`.gitignore`	`.env`, data, PDFs

Three files carry everything.

`README.md`: question, design, data, people. Stable facts.

`CLAUDE.md`: what the agent may and may not do, your conventions. Read at every start.

`PROJECT_HISTORY.md`: dated log. Done, decided, *verified by a human*, open.

The rest is just folders. Copy the template, `git init`, and the agent knows the rules before you say a word.

Guardrails

Written in `CLAUDE.md`

- `data/raw/` is read-only. Always.
- Never invent a reference, a number, a result. Not in the folder = "not found".
- Never delete. Never push. Nothing leaves the folder unless you asked.
- Every claim about a paper cites file and page.

Written in your habits

- API keys live in `.env`; `.env` is in `.gitignore`.
- Confidential data: a sample, anonymised, or never near an API.
- One agent at a time. Read the diff.

A rule in a file is a request, not a guarantee. You will see one fail today.

What You Leave With at Noon

A git folder for your project, containing:

- `README.md` with your question, in your words
- `CLAUDE.md` with your conventions
- 3 to 5 literature notes, one verified by you against the PDF
- a one-page research memo
- a hostile critique of that memo, and your response
- an entry in `PROJECT_HISTORY.md` and a first commit

And one thing you will have seen: the agent making a mistake in a literature note. A wrong number, a wrong page, a finding that is not in the paper. Finding it is step 5 of the worksheet. It is the only step you cannot skip.

No project yet, or no PDFs today? Use the example project. You lose nothing.

Installation Clinic

9:30 – 9:50

Worksheet, step 0, then step 1

Stuck? Paste the error in the shared document and ask your neighbour.

Live Demo

9:50 – 10:25

Who votes far right? The local rise of the FN/RN

The Demo in Seven Steps

1. Empty folder, template, `git init`, start the agent
2. Fill the context (prompt 01)
3. Literature notes from the PDFs (prompt 02)
4. Verify a note against the PDF
5. Research memo (prompt 03)
6. Hostile reviewer (prompt 04)
7. Log, commit, close; ask a fresh session where we stand (prompt 05)

All prompts are in `template/prompts/`. You will paste the same ones on your project.

Hands-on

10:40 – 11:40

Worksheet, steps 1 to 8. Your project, your PDFs.

Steps 1–3 first. Step 5 is mandatory.

Stuck for five minutes? Paste the error in the shared document and ask your neighbour.

Debrief

Three screens. A `CLAUDE.md`. A literature note, with the error that was found. A critique, with the response.

The errors we found today. Read from the shared document. The list is the lesson.

Next Session: 28 October 2026

Session 2: Structured Data with an Agent

- Bring your dataset as a file the agent can open (CSV, Excel, Stata, SPSS, Parquet). Confidential: a sample. Large: a subset.
- No dataset: you will use the example pack (French election results and local indicators, by municipality).
- Keep working in your folder. End every session with prompt 05.

Carry-over question

Next time the agent will clean and merge your data. How would you know that a merge it wrote silently dropped a tenth of your observations?

References

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *METR*, arXiv:2507.09089.

Cunningham, S. (2026). Do AI Agents Writing Full Manuscripts at the Social Catalyst Lab P-Hack? *causalinf.substack.com*.

Ning, L. (2026). The Ideation Bottleneck: Decomposing the Quality Gap Between AI-Generated and Human Economics Research. arXiv:2604.03338.

Social Catalyst Lab (2026). Autonomous Policy Evaluation (APE). `https://ape.socialcatalystlab.org`.

Spence, M. (1973). Job Market Signaling. *Quarterly Journal of Economics*, 87(3), 355–374.